ASSISTÊNCIA MANAGER

Documento de produto, arquitetura técnica e plano de implementação do MVP

PRODUTO	MODELO COMERCIAL	PLATAFORMA
Gestão de assistências técnicas	Licença de pagamento único	Windows desktop • offline

Versão do documento: 1.0 | Data: 12/10/2026

Status: proposta arquitetural inicial, sujeita a validação de implementação e testes.

Finalidade: documento de referência permanente para orientar o Codex, validar entregas e preparar uma versão comercial confiável.

1. Visão geral do produto

O Assistência Manager será uma aplicação para pequenas assistências técnicas de celulares, notebooks, computadores e equipamentos eletrônicos. Seu fluxo central é receber um equipamento, registrar a ordem de serviço (OS), executar o reparo, controlar peças e pagamentos e entregar um comprovante profissional ao cliente.

Público e proposta de valor

- Público inicial: assistências técnicas pequenas, com um operador e um computador Windows.
- Proposta: controlar operações cotidianas com rapidez, sem depender de conexão à internet.
- Distribuição: instalador Windows, manual de uso e política de atualizações/suporte claramente descrita.
- Monetização: pagamento único por uma licença individual; política de múltiplos computadores será definida antes da venda.

Escopo que fica de fora do MVP

Sem emissão de NF-e/NFS-e, integração bancária, sincronização entre PCs, operação multiusuário simultânea, API pública, WhatsApp automático, aplicativo móvel, SaaS ou recursos pagos de IA. São itens de roadmap, não promessas para a primeira venda.

2. Princípios de qualidade

- Sem perda silenciosa de dados: persistência em pasta de usuário, transações e estratégia de atualização segura.
- Sem configuração técnica do comprador: Python e dependências empacotados no instalador.
- Sem acoplamento da UI ao banco: cada camada tem uma responsabilidade definida.
- Sem simular funcionalidade inexistente: cada módulo será liberado após evidência de teste.
- Privacidade e conformidade: coletar apenas informações necessárias, oferecer proteção e transparência no tratamento de dados de clientes.

3. Arquitetura escolhida

Modelo desktop em camadas, executado integralmente no computador do comprador. Não exige hospedagem nem serviço de autenticação online no MVP.

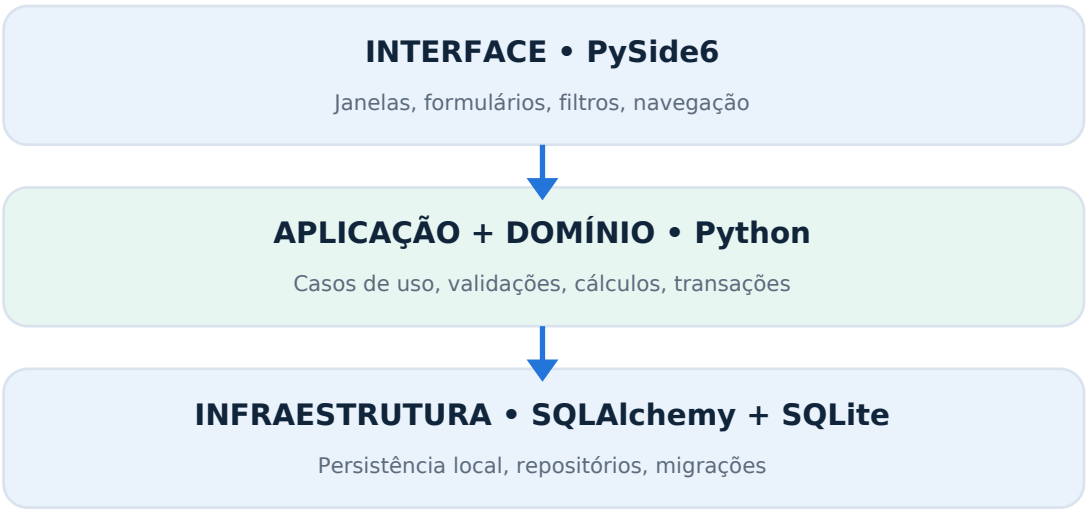


Figura 1 — Camadas principais do aplicativo desktop.

Tecnologia	Papel técnico
Python 3.13	Linguagem e regras do sistema
PySide6 (Qt)	Interface nativa desktop Windows
SQLite + SQLAlchemy 2.x	Banco local e acesso transacional
Alembic	Evolução versionada do esquema do banco
ReportLab	PDF de OS, recibos e relatórios
Pytest	Testes unitários e integração
PyInstaller + Inno Setup	Executável e instalador Windows

4. Funcionamento detalhado do MVP

4.1. Clientes e equipamentos

O operador cadastra clientes e seus aparelhos. Um cliente pode possuir vários equipamentos, e o mesmo equipamento pode ter um histórico de ordens de serviço. Dados opcionais devem permanecer opcionais; busca por nome, telefone ou identificador.

4.2. Ordem de serviço (OS)

A OS registra identificador sequencial, abertura, cliente, equipamento, defeito relatado, acessórios recebidos, diagnóstico, orçamento, peças e serviços, aprovações, execução, garantia, entrega e PDF. O valor total deve ser calculado a partir dos itens e descontos com regra de arredondamento consistente.

Fluxo sugerido: Aberta → Em análise → Aguardando aprovação → Aprovada → Em execução → Concluída → Entregue. Cancelada é um estado terminal com motivo obrigatório. Transições fora da sequência exigem regras explícitas; alterações de status devem ser auditáveis.

4.3. Estoque

Cadastrar peça/produto com SKU, descrição, unidade, custo, preço, quantidade e mínimo. Toda entrada/saída deve produzir uma movimentação com referência e motivo; operações não podem produzir saldo negativo por acidente. Baixa por venda e aplicação em OS exigem idempotência e rollback em caso de falha.

4.4. Vendas e pagamentos

Venda de peças e serviços, itens, desconto, total e forma de pagamento; registrar valores efetivamente recebidos separadamente dos totais previstos. Estorno/cancelamento deve refletir estoque e relatórios conforme regras explícitas. Não equivale a sistema fiscal nem emite documentos fiscais.

4.5. Dashboard, relatórios e configuração

Indicadores: OS abertas, concluídas, entregues, vendas registradas, recebimentos por período e produtos sob mínimo. Personalização da loja (nome, telefone, logo, endereço) nos PDFs. Filtros de data e transparência sobre o significado de cada indicador.

4.6. Backup e restauração

Backup manual + rotina automática configurável. Usar API de backup do SQLite ou procedimento consistente equivalente, validar integridade do artefato, registrar data/versão e testar restauração. Antes de restaurar, guardar cópia de segurança do banco atual e exigir confirmação explícita.

5. Modelo inicial de dados

Modelo lógico de partida. Chaves primárias são IDs estáveis; relacionamentos e restrições devem ser revisados durante a Etapa 01. Valores financeiros são armazenados em centavos inteiros (ou Decimal adequadamente configurado), nunca float binário.

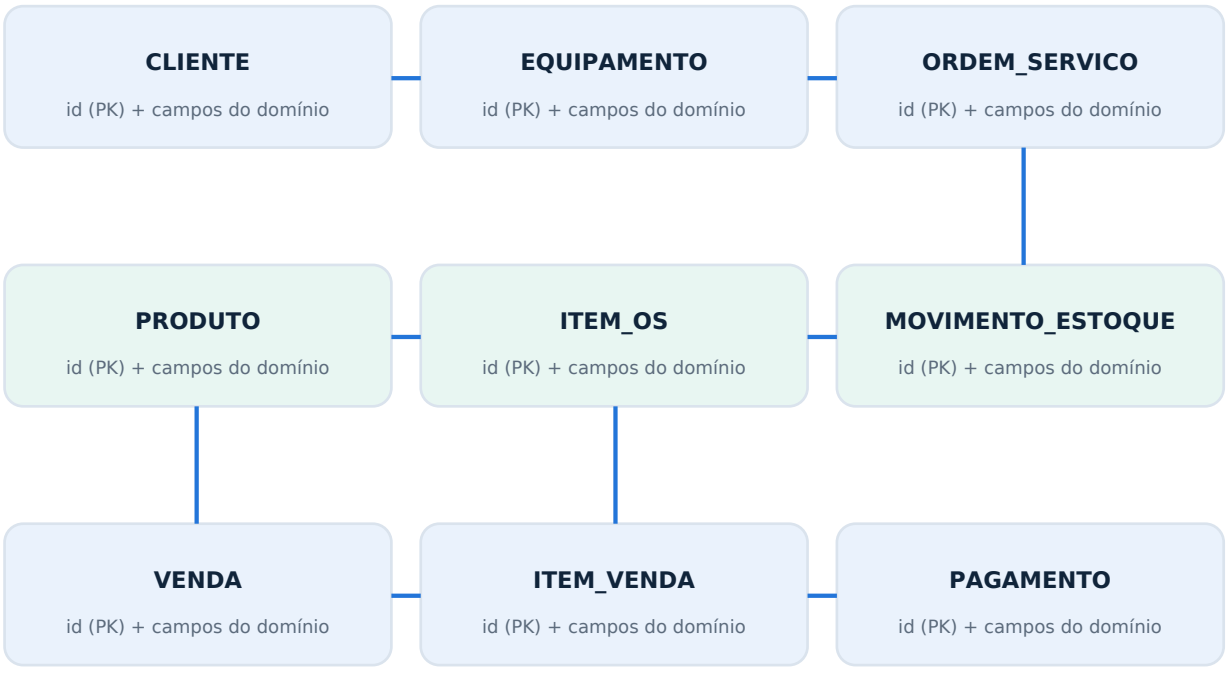


Figura 2 — Diagrama conceitual de entidades (ligações ilustrativas; não representa todas as FKs).

Entidade	Campos e relações centrais
cliente	id; nome; telefone; documento opcional; contato; timestamps. 1:N equipamentos e OS.
equipamento	id; cliente_id; tipo; marca; modelo; serial opcional; observações. 1:N OS.
ordem_servico	id; número único; cliente_id; equipamento_id; status; relato; diagnóstico; desconto; datas; garantia; totais.
item_os	id; ordem_id; produto_id opcional; descrição; categoria serviço/peça; quantidade; valor unitário.
produto	id; sku único; descrição; custo; preço; quantidade; mínimo; ativo.
movimento_estoque	id; produto_id; quantidade assinada; tipo; referência; data; operador; chave de idempotência quando necessária.
venda + item_venda	venda: id/status/totais/data; item: venda_id; produto_id opcional; quantidade; valor.
pagamento	id; venda_id ou ordem_id (vínculo validado); forma; valor; data; status.
configuracao + backup_log	dados da loja, versão do schema, preferências de backup e histórico operacional.

Índices recomendados: OS por número/data/status, cliente por nome/telefone, produto por SKU, movimentações por produto/data e vendas por data. Integridade referencial habilitada no SQLite; exclusão em cascata proibida para histórico comercial sem justificativa.

6. Organização das pastas

```
assistencia-manager/
|-- AGENTS.md
|-- pyproject.toml
|-- README.md
|-- src/assistencia_manager/
|   |-- main.py
|   |-- config.py
|   |-- presentation/
|   |   |-- windows/  widgets/  dialogs/  themes/
|   |-- domain/
|   |   |-- entities/  rules/  exceptions/
|   |-- application/
|   |   |-- services/  dto/
|   |-- infrastructure/
|   |   |-- database/
|   |   |   |-- models/  repositories/  migrations/
|   |   |-- pdf/  backup/  filesystem/
|   |-- resources/icons/
|-- tests/
|   |-- unit/  integration/  ui/
|-- packaging/
|   |-- pyinstaller/  inno_setup/
|-- scripts/
|-- docs/
|   |-- Documentacao_Assistencia_Manager_MVP.pdf
|   |-- PROMPT_INICIAL_CODEX.md
|   |-- arquitetura/  decisoes/  manuais/
|-- .gitignore
```

Convenção: o projeto já existe no computador do desenvolvedor. Criar somente a subpasta docs/ e salvar ali a documentação; não recriar a pasta raiz e não modificar nem apagar conteúdos preexistentes.

7. Plano de implementação por etapas

Etapa	Entregáveis	Critério de aceite
01 — Fundação	AGENTS.md; pyproject; ambiente virtual; pastas; modelos básicos; Alembic; janela; menu; testes.	Janela abre; banco persiste após reinício; testes executam.
02 — Clientes	CRUD clientes/equipamentos; validação; busca; histórico.	Cadastrar, editar, localizar e reabrir sem perder dados.
03 — OS	Estados, orçamento, itens, valores, PDF e histórico.	Percorrer abertura → entrega; validar transições e totais.
04 — Estoque e vendas	Movimentações, inventário, vendas, pagamentos, cancelamento.	Sem baixa duplicada; rollback confiável; testes de valores.
05 — Gestão e segurança	Dashboard, relatórios, personalização, backup/restauração.	Restaurar backup em base de teste e conferir integridade.
06 — Release	PyInstaller, Inno Setup, manual, testes de instalação/upgrade.	Instalar sem Python em Windows limpo; dados sobrevivem a upgrade.

8. Critérios globais de conclusão

- Cada etapa tem testes unitários + integração e, quando necessário, testes manuais de interface.
- Mensagens de erro orientam o usuário sem expor stack traces na interface.
- Nenhuma ação destrutiva ocorre sem confirmação e, quando relevante, backup prévio.
- Logs com dados pessoais minimizados; backups protegidos por permissões do sistema.
- Teste de restauração real, atualização entre versões, instalação limpa e desinstalação sem exclusão inesperada de dados.
- Testar em Windows 10/11 suportados, inclusive diferentes resoluções e configurações regionais brasileiras.
- Revisar licenças do Qt/PySide6 e das bibliotecas, política de uso e instruções de distribuição antes da publicação.

9. Modelo comercial e entrega

Venda prevista: licença individual de uso em um computador Windows, com pagamento único. O preço e a política de transferência da licença devem ser estabelecidos antes de anunciar. Não anunciar “atualizações vitalícias” ou “suporte ilimitado” sem capacidade operacional.

Pacote entregue: instalador assinado digitalmente se viável, notas de versão, guia de instalação, informações de backup, termo de licença e instruções de suporte. Assinatura de código é recomendável, embora possa representar custo.

Mercado Livre: conferir as regras atuais da categoria e modalidades aceitas para software digital no momento da publicação. Não presumir que envio de link, chave ou arquivo por mensagens sempre é autorizado. A entrega e comprovação precisam seguir as regras vigentes.

Segurança e responsabilidade

Licença perpétua não é o mesmo que atualizações ilimitadas. O MVP offline single-user não oferece sincronização nem colaboração. O armazenamento de informações de clientes exige avisos de privacidade e medidas razoáveis de proteção em conformidade com a LGPD. Planejar exportação de dados e instruções para backup do operador.

10. Governança do Codex

O agente trabalha em incrementos pequenos, respeita limites de escopo, registra decisões e reporta evidência de teste. O arquivo AGENTS.md deverá institucionalizar estas regras no repositório.

- Inspecionar o projeto existente antes de criar ou alterar arquivos.
- Explicar alternativas e riscos técnicos antes de decisões importantes.
- Pedir aprovação para instalar ferramentas globais, eliminar dados, fazer releases ou mudar arquitetura.
- Não implementar etapas futuras sem autorização; não refatorar módulos estáveis sem justificativa.
- Ao concluir: resumo do que foi feito; arquivos alterados; decisões; testes executados e falhas conhecidas.

11. Prompt inicial para o Codex (copiar integralmente)

O texto abaixo também acompanha esta entrega como arquivo PROMPT_INICIAL_CODEX.md. A primeira tarefa deve ser planejamento e não a implementação imediata de todos os módulos.

Projeto Assistência Manager - início com Codex

Você será o engenheiro de software responsável pelo Assistência Manager, sistema desktop Windows de gestão de assistências técnicas, vendido por pagamento único. A pasta raiz do projeto JÁ EXISTE. Inspecione-a antes de qualquer alteração e não recrie a raiz.

Objetivo e escopo Aplicação offline, single-user, sem servidor nem assinatura no MVP. Funcionalidades planejadas: clientes e equipamentos; OS com estados, orçamento, serviços, peças e PDF; estoque com movimentações; vendas e pagamentos; dashboard; configurações da loja; backup e restauração. Fora do MVP: NF-e/NFS-e, WhatsApp automático, IA, nuvem, sincronização multi-PC.

Stack e arquitetura Python 3.13, PySide6, SQLAlchemy 2.x, SQLite, Alembic, ReportLab, Pytest, PyInstaller e Inno Setup. Usar arquitetura em camadas: presentation, application, domain, infrastructure; src-layout. Antes de fixar versões, confirmar compatibilidade e licenças comerciais de dependências, especialmente Qt/PySide6. Manter UI desacoplada de SQL e regras de negócio.

Dados e confiabilidade Salvar base e backups em diretórios persistentes de usuário Windows, nunca dentro do executável ou da pasta de instalação. Habilitar integridade referencial. Usar transações atômicas para estoque, OS, vendas e pagamentos, evitar baixa duplicada e tratar cancelamentos. Não usar float para valores monetários. Criar migrations desde o começo. Backups consistentes do SQLite e restauração testada, mantendo cópia prévia e pedindo confirmação. Não usar dados reais em testes.

Estrutura-alvo AGENTS.md, pyproject.toml, README.md, src/assistencia_manager/{main.py,config.py,presentation/,domain/,application/,infrastructure/,resources/}, tests/{unit/,integration/,ui/}, packaging/{pyinstaller/,inno_setup/}, scripts/, docs/. Preservar o que já existir. Dentro de docs/ guardar os documentos do produto, decisões e manuais.

Qualidade Interface profissional e utilizável em 1366x768. Telas sem travar em tarefas demoradas. Tipagem, nomes claros e logs sem dados sensíveis. Testes automatizados unitários e de integração; não declarar sucesso sem evidência. Instalação Windows sem Python deverá ser validada antes da etapa comercial. Documentar limitações.

Como você deve trabalhar 1. Antes de codificar, inspecione arquivos existentes e apresente arquitetura, árvore de pastas, dependências/versões, riscos e critérios de aceite. 2. Explique opções e riscos das decisões importantes; peça minha aprovação. 3. Trabalhe em incrementos pequenos; NÃO faça as etapas 02 a 06 sem autorização. 4. Nunca exclua conteúdo existente ou altere arquivos externos ao

projeto sem consentimento. 5. Não instale ferramentas globais sem aprovação; use venv. 6. Execute e relate testes relevantes após mudanças. 7. Entregue sempre resumo do feito, arquivos alterados, decisões técnicas, testes feitos/não feitos e pendências. 8. Mantenha AGENTS.md, README.md e docs/ consistentes com as decisões aprovadas.

Etapa 01 - Fundação (somente após minha aprovação) Criar estrutura inicial, pyproject, venv local, AGENTS.md, configuração do pytest, SQLite + SQLAlchemy, scaffolding Alembic, janela principal PySide6, menu lateral de módulos ainda não implementados e smoke tests de inicialização/persistência. Não implementar CRUD, OS, estoque ou vendas nesta etapa.

Sua primeira resposta deve trazer SOMENTE o diagnóstico do diretório, plano e decisões a

aprovar; não codifique até eu autorizar. Leia a documentação em docs/ se estiver disponível.

12. Próximos passos

- Salvar este PDF dentro de assistencia-manager/docs/ e manter o documento sob controle de versão.
- Abrir o Codex a partir da pasta raiz e enviar o arquivo PROMPT_INICIAL_CODEX.md ou colar seu conteúdo.
- Aprovar as decisões da Etapa 01 antes de permitir implementação.
- Validar critérios de aceite ao final de cada etapa e atualizar esta documentação quando decisões mudarem.

Documento de referência técnica e comercial inicial. Alterações de escopo devem ser versionadas; nenhuma estimativa comercial representa garantia de vendas.